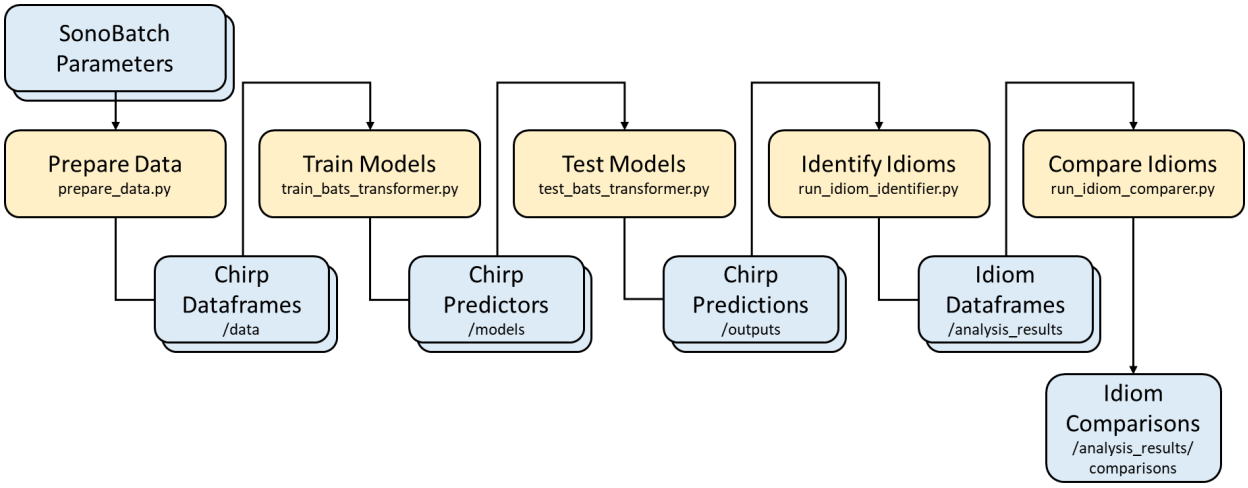
BatFormer + Idiom Analysis Pipeline

Andrew Chen, March 2026, v1.0

This document explains how to use our next bat chirp predictor and subsequently run analyses on the results. Each step is a command-line script to run from the bats/bats_transformer repo.



A Note on Reading The Sample Filesystems

In this guide, I include sample filesystems so you know what to expect to see after running each part. The folders and files are formatted in the sample filesystems such that folders are immediately preceded by a "|" and files are immediately preceded by a "-". Each level of the filesystem is marked by a "| - ". Ellipses ("...") are used for obvious similar sets of files/folders and also follow the formatting rules. For example,

```
None
| root_folder
| - | subfolder_1
| - | - | subsubfolder_1a
| - | - file_1_1
| - | - ... (file_1_2 through file_1_9)
| - | - file_1_10
| - | subfolder_2
| - | - | subsubfolder_2a
| - | - | ... (subsubfolder_2b through subsubfolder_2d)
| - | - | subsubfolder_2e
| - | - file_2_1
| - root_file_1
```

Part 1:

This part is adapted from Varun Desai's documentation.

Setting up the workspace

The repo can be found at: https://github.com/paepcke/bats.

To clone, run

```shell
git clone https://github.com/paepcke/bats.git
cd bats && git submodule init
git submodule update
cd bats_transformer
conda create -f env310.yml
cd spacetimeformer && pip install -e .
cd ../.. && pip install -e .
```

The critical components are:

```
| bats
| - | bats_transformer
| - | - | scripts
| - | - | spacetimeformer
| - | src
| - | - | bats_dataset
| - | - | data_calcs
| - | - | idiom_analysis
| - | - | peak_detection
```

The data

I assume that at this point, you will have a folder containing SonoBatch summary files consisting of chirp attributes. Specifically, the folder should have the following structure:

```
| all_data
| - | day_1-4_data
```

```
| - | - | day_1_Parameters_etc
| - | - | ...
| - | - | day_4_Parameters_etc
| - | day_5-8_data
| - | - | day_5_Parameters_etc
| - | - | ...
| - | - | day_8_Parameters_etc
| - | ...
```

The following script will look at the outermost level (all_data), and then glob all files that have *_Parameters_* in the filename (these should be outputted from SonoBatch).

From the project root (bats), run

Shell

```
python src/bats_dataset/prepare_data.py \
       -i /qnap/bats/barn_sonobat3_2_processed \
       -o datasets/2022_barn_Myca_quantile/splits \
       -s 10 \
       -f \
       -m 5 \
       --scaler quantile \
       --species Myca
```

This outputs a folder containing .feather files, representing the prepared dataset used for training/prediction.

None

```
| datasets
| - | 2022_barn_Myca_quantile
| - | - | splits
| - | - | - split_config.csv
| - | - | - split_filename_to_id.csv
| - | - | - split_mapping.csv
| - | - | - split_scaler.pkl
| - | - | - split_truth_values.feather
| - | - | - split0.feather
| - | - | - ...
| - | - | - split9.feather
```

Command-Line arguments:

-i INPUT_DATA_PATH, --input_data_path INPUT_DATA_PATH
    Folder containing subfolders with SonoBatch Parameters files
-o OUTPUT_DATA_PATH, --output_data_path OUTPUT_DATA_PATH
    Folder for dataset and metadata
-s SPLITS, --splits SPLITS
    Number of splits to shard dataset
-f, --use_feather
    Use .feather format (instead of .csv)
-m MINIMUM_LENGTH, --minimum_length MINIMUM_LENGTH
    Minimum sequence length to accept
-d, --daytime
    Only keep files starting before sunset (instead of all files)
--species SPECIES
    Filter audio files by four-letter species code
--scaler SCALER
    Which scaler to use to normalize data

Training the models

At this point, you will have a folder containing the dataset in .feather format. See the BatFormer Documentation for a more detailed description of what is going on in the model. Importantly, because of the way these scripts were originally set up, there are some command-line arguments that do not make their way up the inheritance chain to this script, and thus must be set in the scripts themselves (this is, of course, a place for future improvement).

From the project root (bats), run

```Shell
python scripts/train_bats_transformer.py  \
       --random_seeds {31..40} \
       --data_path datasets/2022_barn_Myca_quantile/splits \
       --model_path models/2022_barn_Myca_quantile
```

This will output a folder containing models.

```None
| models
| - | 2022_barn_Myca_quantile
| - | - model_31
| - | - ...
```

```
| - | - model_40
```

Command-line arguments:

--random_seeds RANDOM_SEEDS [RANDOM_SEEDS ...]
    Random seeds to use for training
--gpu GPU
    GPU to use for training
--data_path DATA_PATH
    Path to the data
--shuffle_data
    Whether to shuffle the data during training
--quantize
    Whether to use quantized model (experimental)
--model_path MODEL_PATH
    Folder to output trained models
--ignore_cols IGNORE_COLS [IGNORE_COLS ...]
    Which chirp attributes to remove from data

Model inference

At this point, you will have a folder containing the dataset in .feather format and a folder containing trained models.

From the project root (bats), run

```Shell
python scripts/test_bats_transformer.py \
       --model_paths models/2022_barn_Myca_quantile/model_{31..40} \
       --out_dir outputs/2022_barn_Myca_quantile_run1/ \
       --input_data_path datasets/2022_barn_Myca_quantile/splits \
       --gpus 0
```

This will output a folder containing .log (essentially .csv) files containing the predictions, ground truths, and losses for each model.

```None
| outputs
| - | 2022_barn_Myca_quantile_run1
```

```
| - | - model_31_ground_truths.log
| - | - model_31_losses.log
| - | - model_31_predictions.log
| - | - ...
| - | - model_40_ground_truths.log
| - | - model_40_losses.log
| - | - model_40_predictions.log
```

Command-line arguments:

--model_paths MODEL_PATHS [MODEL_PATHS ...]
    Folder containing trained models
--quantized
    Whether the model is quantized
--ignore_cols IGNORE_COLS [IGNORE_COLS ...]
    Which chirp attributes to remove from data
--out_dir OUT_DIR
    Folder to output generated results
--input_data_path INPUT_DATA_PATH
    Folder to dataset containing chirp attributes
--shuffle_data
    Whether to shuffle the data during testing (should match training)
--gpus GPUS
    Number of GPUs to use

Part 2:

Idiom identification

At this point, you will have a folder containing prediction, ground truth, and loss .log files for each model. You will also still have the folder containing the dataset used to generate those predictions in .feather format.

From the project root (bats), run

```shell
python src/idiom_analysis/run_idiom_identifier.py
       --output_folder outputs/2022_barn_Myca_quantile_run1
       --dataset_path datasets/2022_barn_Myca_quantile/splits
```

```
    --results_folder analysis_results
```

This will output a folder containing various dataframes about the idioms identified.

```
None
| analysis_results
| - | 2022_barn_Myca_quantile_run1
| - | - | figs
| - | - | - ...
| - | - all_chirp_measures_scaled_robust.csv
| - | - all_chirp_measures.csv
| - | - chirp_prediction_confidence_measures.csv
| - | - idiom_boundaries.csv
| - | - test_set_chirp_attributes.csv
```

Command-line arguments:

From run_idiom_identifier.py

--output_folder OUTPUT_FOLDER
    Path to the output folder containing the data to analyze
--dataset_path DATASET_PATH
    Path to the dataset csv file containing the chirp attributes
--results_folder RESULTS_FOLDER
    Path to the folder where results will be saved

From IdiomIdentifier

--measure MEASURE
    Which uncertainty measure to use: ["radius_mean", "density", "tightness"]
--prediction_offset PREDICTION_OFFSET
    First chirp index in test set sequences, i.e. minimum context size
--sigma SIGMA
    Sigma value for Gaussian smoothing
--low_conf_percentile LOW_CONF_PERCENTILE
    Percentile at which to declare low confidence

From ChirpClusterer

--no_amp NO_AMP
    Whether to include chirp measures involving "Amp"
--min_cluster_k MIN_CLUSTER_K

Minimum k to consider in determining optimal k
--max_cluster_k MAX_CLUSTER_K
Maximum k to consider in determining optimal k
--reduc_method REDUC_METHOD
Which method to use to reduce data to 2-D visualization: ["umap", "pca"]
--cluster_method CLUSTER_METHOD
Which method to use for clustering: ["Agglomerative", "HDBSCAN"]
--calculate_k
Whether to calculate optimal k (instead of using --cluster_k)
--cluster_k CLUSTER_K
Fixed number of clusters (k)

From SubseqCounter

--subseq_n SUBSEQ_N
Length of subsequences to count
--subseq_k SUBSEQ_K
Number of most common subsequences to output
--subseq_type SUBSEQ_TYPE
What type of subsequence: ["all", "prefix", "suffix"]
--subseq_calc_prob SUBSEQ_CALC_PROB
Whether to calculate probabilities (in addition to absolute counts)

Idiom comparison

After running the previous steps twice or more on different datasets, you will have at least two folders containing various dataframes about the idioms identified in the analysis.

From the project root (bats), run

```shell
python src/idiom_analysis/run_idiom_comparer.py
       --exp_1_folder analysis_results/2022_barn_Myca_quantile_run1
       --exp_2_folder analysis_results/2022_lake_Myca_quantile_run1
       --exp_1_name Barn
       --exp_2_name Lake
       --results_folder analysis_results/comparisons
```

This step currently does not return any dataframes, but will generate figures and place them in a new folder.

```
None
| analysis_results
| - | 2022_barn_Myca_quantile_run1
| - | - ...
| - | 2022_lake_Myca_quantile_run1
| - | - ...
| - | comparisons
| - | - | Barn_Lake
| - | - | - | figs
| - | - | - | - ...
```

Command-line arguments:

From run_idiom_comparer.py

--exp_1_folder EXP_1_FOLDER
    Path to the first set of analysis results
--exp_2_folder EXP_2_FOLDER
    Path to the second set of analysis results
--exp_1_name EXP_1_NAME
    Name of the first experiment (for labeling purposes)
--exp_2_name EXP_2_NAME
    Name of the second experiment (for labeling purposes)
--results_folder RESULTS_FOLDER
    Folder to save comparison results and figures

From ChirpClusterer

--no_amp NO_AMP
    Whether to include chirp measures involving "Amp"
--min_cluster_k MIN_CLUSTER_K
    Minimum k to consider in determining optimal k
--max_cluster_k MAX_CLUSTER_K
    Maximum k to consider in determining optimal k
--reduc_method REDUC_METHOD
    Which method to use to reduce data to 2-D visualization: ["umap", "pca"]
--cluster_method CLUSTER_METHOD
    Which method to use for clustering: ["Agglomerative", "HDBSCAN"]
--calculate_k
    Whether to calculate optimal k (instead of using --cluster_k)
--cluster_k CLUSTER_K
    Fixed number of clusters (k)

From SubseqCounter

--subseq_n SUBSEQ_N
    Length of subsequences to count
--subseq_k SUBSEQ_K
    Number of most common subsequences to output
--subseq_type SUBSEQ_TYPE
    What type of subsequence: ["all", "prefix", "suffix"]
--subseq_calc_prob SUBSEQ_CALC_PROB
    Whether to calculate probabilities (in addition to absolute counts)

Conclusion

At the end of the day, the workspace should look something like this:

```
None
| bats
| - | analysis_results
| - | - ...
| - | - | comparisons
| - | - | - ...
| - | all_data
| - | - ...
| - | bats_transformer
| - | - | scripts
| - | - | spacetimeformer
| - | datasets
| - | - ...
| - | models
| - | - ...
| - | outputs
| - | - ...
| - | src
| - | - | bats_dataset
| - | - | data_calcs
| - | - | idiom_analysis
| - | - | peak_detection
```